COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure & Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	Oct 2, 2026	**DATE SUBMITTED:**	Oct 3, 2026
NAME:	Ralph Angelov F. Braganza	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

CODE:

main.cpp

main.cpp > DiaGraph

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }
    int N; // number of nodes in the graph

public:
    adjNode **head; //adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;

        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
```

```
            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++) {
            adjNode* current = head[i];

            while (current != nullptr) {
                adjNode* temp = current;
                current = current->next;
                delete temp;
            }
        }

        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
                  << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);

    // construct graph
```

```
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;

    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

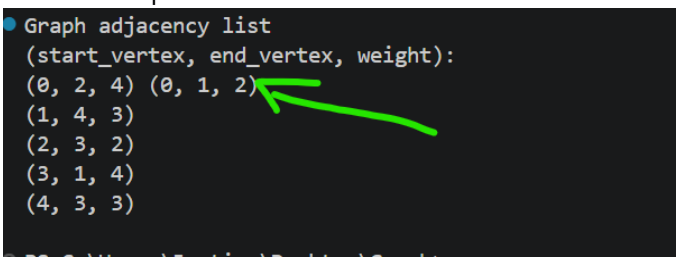
OUTPUT

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ANALYSIS:

The structure used in the creation of entries of the adjacency list is called **adjNode**. It contains three fields: **val** (used to store the destination vertex), **cost** (used to store the edge cost), and **next** (used to point to the next node).

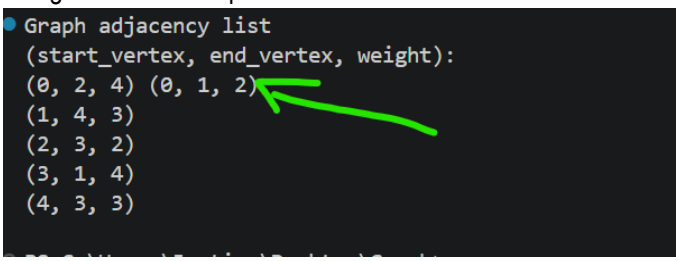
One of the outputs is:



It means vertex 0 connects to vertex 1 with a weight of 2.

The **graphEdge struct stores the original edge information, including the starting vertex, destination vertex, and weight.** Unlike adjNode, which represents connections inside the adjacency list, graphEdge is mainly used as the input when constructing the graph.

Going back to our output



0 is the starting vertex, then our ending vertex is 1 and the weight is 2.

The **DiaGraph class is responsible for building and managing the directed graph.** It uses an array of pointers called head, where each position represents a vertex and points to the beginning of its adjacency list. Since the graph is directed, each edge only creates a connection from its starting vertex to its destination, rather than automatically creating a reverse connection.

getAdjListNode function dynamically allocates space for adjacency list nodes with the help of new operator. Each newly created node is inserted at the head of the linked list, and it's done by pointing its next field towards the previous head node.

The **constructor initializes the graph by allocating memory for the array of head pointers and setting each position to nullptr.** It then processes the given edges and inserts them into their corresponding adjacency lists. Since new nodes are inserted at the beginning, the displayed order of connections is the reverse of their insertion order for each vertex.

The **destructor is responsible for releasing the dynamically allocated memory.** After careful analysis I've observed that it wasn't freeing the allocated memory properly. **Originally it used delete[] head[i], which was incorrect because each linked-list node was allocated individually using new, not as an array.** We changed it so that the destructor first traverses each linked list and deletes its nodes one by one using delete. After deleting all the nodes, it uses delete[] head to free the array of pointers. This prevents memory leaks and ensures that the allocated memory is properly released.

The **display_AdjList() function prints out the connections of a specific vertex along with their destination**

and edge weight. The linked list is traversed till nullptr is reached. It becomes easy to inspect how the graph is stored using its adjacency list.

The main() function defines the graph's edges, determines the number of edges, creates the DiaGraph object, and displays the adjacency list of each vertex. This function gives an idea about how the graph is being constructed using the edge details. After completion of the program execution, the destructor takes care of memory deallocation.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search CODE

mainILOB1.cpp

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Step 2. Write the following struct in order to implement an edge in our graph:
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3. Next, overload the << operator for the graph so that it can be printed out using the following function:
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

// Step 4. Implement the graph data structure that uses an edge list representation as follows:
```

```cpp
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
```

```
        return edges_from_v;
    }

    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

// Step 5. Now, we need a function to perform DFS for our graph. Implement it as follows:
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if (visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

// Step 6. We shall test our implementation of the DFS on the graph shown here:
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

// Step 7. Finally, add the following test and driver code, which runs our DFS implementation and prints the output:
template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

OUTPUT:

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

ANALYSIS:

The **Graph class** is responsible for handling the vertices and edges of the graph using an **Edge List Representation**. In contrast to having a separate list of edges for each vertex, it uses a single vector of edges called **edge_list**. The **struct Edge** forms the backbone for creating links between vertices in the graph. The **source vertex, destination vertex, and edge weight are stored in this structure.** With the overloaded comparison operators, edges can be compared according to their weights. Even though in this algorithm **we have set all weights to be zero**, the weights **in the structure are kept in case they are needed in future algorithms.**

Moving forward, the **add_edge() function** is responsible for **adding edges into the graph**. Before doing so, it performs the check if the vertices from which and to which we want to add edges belong to the proper range of vertex numbers. In this way, we do not add any improper vertex number into our graph. The edge is added via **emplace_back()** to **insert the edge into the vector**, allowing the graph to build its connections dynamically.

The **outgoing_edges() function** retrieves **all the edges that originate from a specific vertex**. It traverses the whole edge list, **comparing the source vertex of each edge with the specified vertex**. Whenever there is a match, the corresponding edge is stored in another vector. The **depth_first_search() function** is the one which executes the traversal process by employing the use of stack, set, and vector. This algorithm starts with vertex 1, then follows unvisited connected vertices and keeps their visiting order. The LIFO principle of the stack helps it explore the deep connections first, and the visited set avoids repeating visits.

The **create_reference_graph() function** builds the reference graph using a map to organize the vertex connections. It uses nested loops and **add_edge()** to insert each connection into the graph. The **test_DFS() function** tests the implementation by creating and displaying the reference graph, running DFS starting from vertex 1, and printing the resulting traversal order. Then finally the **main() function** starts the program by defining the template type as unsigned and calling **test_DFS()**, which executes the entire implementation.

B.2. Breadth-First Search CODE

mainILOB2.cpp

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

// Step 2: Write the following struct, which represents an edge in our graph:
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:
template <typename U>
std::ostream &operator<<(std::ostream &os, const Graph<U> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

// Step 4: Write a class to define our graph data structure, as shown here:
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }

    auto &edges() const
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Step 5: For this exercise, we shall test our implementation of BFS on the following graph:
template <typename T>
```

```cpp
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

// Step 6: Implement the breadth-first search like so:
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
```

```cpp
// Step 7: Add the following test and driver code that creates the reference graph, runs BFS starting from vertex
// 1, and outputs the results:
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
}

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

OUTPUT

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

ANALYSIS

The **Edge struct** is responsible for the connections between vertices by **storing the source, destination, and weight**. This is very similar to DFS because it includes comparison operators for comparing edge weights. However, these operators are not directly used by BFS because the algorithm **focuses on the order of traversal rather than edge weights.**

The **Graph class** manages the vertices and connections using and stores all edges in a vector called edge_list. This is the same structure used in DFS, allowing both algorithms to work with the same graph representation.

However, while constructing the original code from step 3 we encountered an issue in the friend function declaration. It reused the **template parameter T**, which was already declared by the **Graph class**. Which causes a template parameter shadowing error. To fix this we used **U instead to avoid this error from happening**. It was also **missing a return statement in the operator<< function**. Without this, it causes undefined behavior when used.

ORIGINAL CODE:

```
31  // Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:
32  template <typename T>
33  std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
34  {
35      for (auto i = 1; i < G.vertices(); i++)
36      {
37          os << i << ":\t";
38          auto edges = G.outgoing_edges(i);
39          for (auto &e : edges)
40              os << "{" << e.dest << ": " << e.weight << "}, ";
41          os << std::endl;
42      }
43  }   no return statement in function returning non-void [-Wreturn-type]
```

```
84      template <typename T>   declaration of template parameter 'T' shadows template parameter [-Wtemplate-body]
85      friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);
```

FIXED CODE:

```
31  // Step 3: Next, overload the << operator for the Graph data type in order to display the contents of the graph:
32  template <typename U>
33  std::ostream &operator<<(std::ostream &os, const Graph<U> &G)
34  {
35      for (auto i = 1; i < G.vertices(); i++)
36      {
37          os << i << ":\t";
38          auto edges = G.outgoing_edges(i);
39          for (auto &e : edges)
40              os << "{" << e.dest << ": " << e.weight << "}, ";
41          os << std::endl;
42      }
43      return os;
44  }
45
```

```
85      template <typename U>
86      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
87
```

The **breadth_first_search() function** does the traversal operation with the help of queue, set, and vector. While DFS traverses by exploring the depth-first connections with the help of a stack, BFS does the same with the help of a queue, which works with the **FIFO principle.** The **create_reference_graph()** function creates the reference graph using a map to organize the connections which is similar to the DFS but the **main difference is that this version assigns different weights to the edges**.

The **test_BFS()** function tests the implementation by creating and displaying the reference graph, running BFS starting from vertex 1, and printing the resulting traversal order. The **main() function** defines the template type as unsigned and calls **test_BFS() to execute the program.**

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.**

   The best algorithm that suits this scenario is Depth First Search (DFS). This algorithm works by starting from one vertex and traversing as far as it can go and then backtracking to the initial vertex and taking up other unvisited connections. In this way, all the reachable vertices are traversed. The reason for this is that the person will need to traverse one route fully before taking another.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**

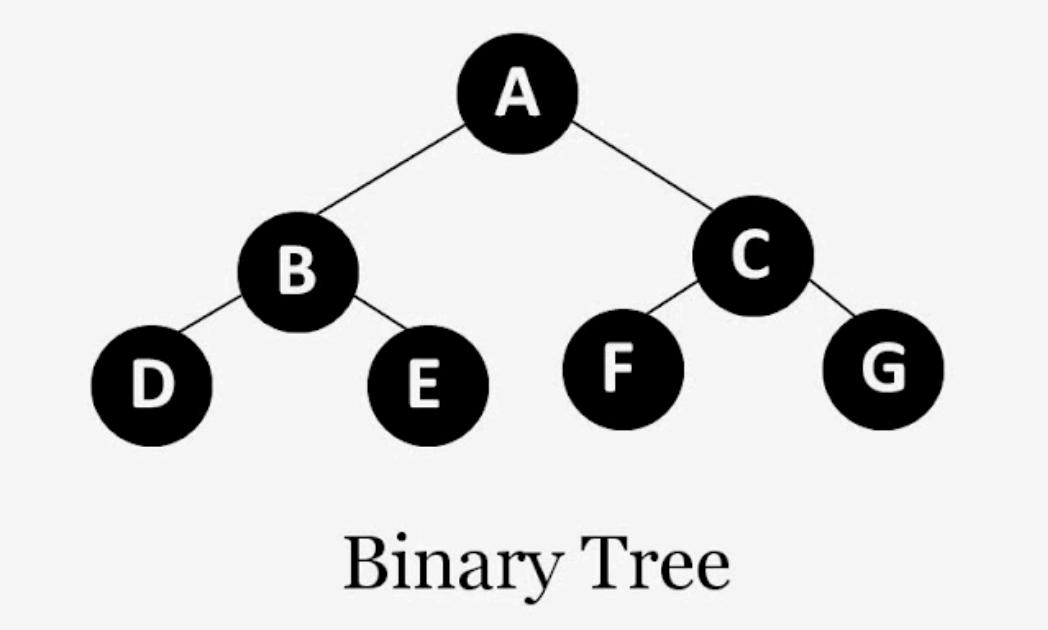
   The equivalent of DFS in tree traversal is Depth-First Tree Traversal. It explores a branch as deeply as possible before moving to another branch. Its three common types are:

   Preorder: Root -> Left ->Right

Inorder: Left-> Root-> Right

Postorder: Left -> Right ->Root

Graphical Comparison



Preorder
A, B, D, E, C, F, G

Inorder
D, B, E, A, F, C, G

Postorder
D, E, B, F, G, C, A

CODE:

mainILOC.cpp > main()

```cpp
#include <iostream>

// Class describing a node of tree
class Node {
public:
    int data;
    Node* left;
    Node* right;

    Node(int v) {
        this->data = v;
        this->left = this->right = nullptr;
    }
};

// Inorder Traversal
void printInorder(Node* node) {
    if (node == nullptr)
        return;

    // Traverse left subtree
    printInorder(node->left);

    // Visit node
    std::cout << node->data << " ";

    // Traverse right subtree
    printInorder(node->right);
}

// Driver code
int main() {
    // Build the tree
    Node* root = new Node(100);
    root->left = new Node(20);
    root->right = new Node(200);
    root->left->left = new Node(10);
    root->left->right = new Node(30);
    root->right->left = new Node(150);
    root->right->right = new Node(300);

    // Function call
    std::cout << "Inorder Traversal: ";
    printInorder(root);
    std::cout << "\n";

    return 0;
```

OUTPUT

```
Inorder Traversal: 10 20 30 100 150 200 300
```

PSEUDO CODE:

```
INORDER(node)

  IF node is NULL
     RETURN

  INORDER(node.left)
  PRINT node.data
  INORDER(node.right)


MAIN
```

```
        CREATE the tree

        INORDER(root)

    END
```

The code implements Inorder Traversal, a type of DFS that follows the Left → Root → Right sequence. It uses recursion to explore each subtree and print the nodes in order. The expected output is 10, 20, 30, 100, 150, 200, 300.

3. **In the performed code, what data structure is used to implement the Breadth First Search?**
   BFS uses a Queue, which follows the FIFO (First In, First Out) principle.

4. **How many times can a node be visited in the BFS**
   BFS, each vertex is visited and processed only once because a visited set keeps track of previously processed vertices.

Conclusion:

From the hands-on activity, I have acquired knowledge about how DFS and BFS algorithms work, and how they use different ways to traverse a graph. I have been able to understand that DFS algorithm uses stack to traverse deep into nodes until it has backtracked to continue traversal, whereas BFS algorithm uses queue to traverse nodes layer by layer. I have understood how the graphs can be represented using Edge list and Adjacency list data structures. Through the whole process, I have become more knowledgeable about how these two data structures interact with graph traversing algorithms. I have also come to learn that it is very important to deal well with dynamically allocated memory so as to avoid memory leaks. During these activities, I have learned how the functions interact to facilitate graph traversal process.

Regarding the supplementary activity, I have come to learn that DFS can also be used to represent tree traversal using different traversal techniques, such as inorder traversal. While analyzing the code, I have understood how recursion works during traversal of tree from left to root to right.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **AI was used for formatting and debugging all the code was already provided in the procedure. It was also used to explain concepts and give a better analysis of the code.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.